

## MP3: Fun with Vectors and Abstraction

### Due Time: Refer to the course web-site

#### Objective

In this programming assignment, we want to verify our understanding of vectors, their applications we have learned in class, and start considering abstraction of behaviors (defining behavior as a class and instantiate at runtime when necessary.)

**Note:** this assignment is for both CSS451 and CSS551

---

#### Approach

We will build a simple simulation application, one where we continuously generate **TravelingBalls** and manipulate **LineSegments** to simulate simple behaviors and interactions between these objects. Here is [my attempt at implementation](#) for your reference.

I will explain the exercise, **hint**, according to the order of my implementation.

First, the **MainCamera**, here are my settings:

- Position: (0, 16.5, -20)
- Rotation: (22, 0, 0)
- Scale: (1, 1, 1)
- Fields of View: 45

Second, the world box, consisting of four perpendicular planes: **LeftWall**, **RightWall**, **BackWall**, and **Floor**. All are basic grey-ish white color. Here are their respective transform settings:

- LeftWall: P(-17, 0, 12), R(-180, 90, 0), S(34, 34, 1) [**Note:** all my walls are Quad]
- RightWall: P(17, 0, 12), R(0, 90, 0), S(34, 34, 1)
- BackWall: P(0, 0, 24), R(0, 0, 0), S(34, 34, 1)
- Floor: P(0, 0, 12), R(90, 0, 0), S(34, 34, 1)

Third, the **AimLine**. This **LineSegment** initially runs from the **LeftWall** towards the **RightWall**. There are corresponding **LineEndPts** that the user can click on and manipulate the **AimLine** endpoints. The **AimLine** endpoints must always stay on one of the four walls of the world box. Here are the details:

- **LineSegment**: you should draw a scaled/rotated cylinder to represent the line segment defined by the endpoints. Display the line segment in grey-ish color.
- **LineEndPoint**: This is a scaled sphere S(2, 2, 2). A user can always left-mouse click on and drag a **LineEndPoint**. Remember, **LineEndPoint** always stay on the walls (Left, Right, Back, or Floor) of the world box.
- **AimLine**: The cylinder width is 0.2 or, S(0.2, LineLength, 0.2)

Fourth, the **TravelingBalls**. These are simple spheres, S(1, 1, 1), that continuously moves from the begin towards the end of an **AimLine**. You must allow your user to control these parameters (please implement these as SlidersWithEcho and place these in one of the *unimportant* corner):

- **Interval**: in seconds upon which a new **TravelingBall** is generated. The range is 0.5 to 4, default 1.
- **Speed**: in units/second where the next **TravelingBall** will travels. The range is 0.5 to 15, default 6.
- **LifeSpan**: in seconds where the next **TravelingBall** will be alive for. The range is 1 to 15, default 10.

When generated, a **TravelingBall** receives the current **AimLine** direction and the **Speed** and **LifeSpan** settings. Note, once generated, the **Speed** and **LifeSpan** parameters of **TravelingBalls** do not change. For this reason, it is entirely possible for a newly generated **TravelingBall**, with a faster speed and longer lifespan, to overtake and out-live previously generated ones.

Fifth, the **BigLine**. Similar to the **AimLine**, the **BigLine** also has two **LineEndPts**. The differences are:

- LineWidth is  $S(1.9, \text{LineLength}, 1.9)$ , just slightly less than of the **LineEndPoint** so that the **LineEndPoint** sphere is always visible.
- As in the case of **AimLine**, the endpoints of **BigLine** will never leave their corresponding walls.
- **TravelingBall** interaction: traveling ball does not travel on big-lines. We will use these lines in the next assignment.
- **Note:** if you think about the design and behavior, AimLine and BigLine are *identical*. If you designed your solution properly, this is literally a few lines of code.

#### World Interaction:

- Here is the logic of the left-mouse button (LMB) click, in priority ordering:
  - If hits a **LineEndPoint**:
    - Selects the end point, turns the selected end point black
    - Drags the end point (and thus manipulate the corresponding line)
    - Until LMB release and turns the end point back to red
  - Else, If hits Left or Back wall
    - Left wall: creates a new horizontal **AimLine** at the (y, z) position from the mouse click
    - Back wall: creates a **BigLine** from (x, y, backwall.z) to (x, 0, -1f), where (x, y) are coordinate values from the mouse click
  - Else, if click on the line segment of any **AimLine** or **BigLine**, the clicked on is removed (including the reflected segment if one existed)
    - In my implementation, you can only delete a line when mouse click is in the *floor* region (e.g., you cannot delete a line on the wall). It is *acceptable* if you have a similar behavior.
  - **Note:** the above means, you cannot delete lines by attempting to click on the line over the Left or Back walls (try it, you will end up creating new AimLine or BigLine).

#### Requirements:

- **Colors:** please use appropriate colors such that all object interactions can be clearly visible.

#### Hints:

1. Please create Resizable Windowed Application for submission (initial resolution: 1920x1080)
2. If, as a result of back-face-culling, you cannot see the walls, rotate them 180-degrees. Remember that, if cannot see an object, RayCast will not work.
3. You are *only allowed* to use RayCast when working with user mouse clicks. The rest of the intersections must be computed by you. That is one of the main objectives of this assignment.
4. I took advantage of the unity **Layer** when decoding LMB
  - a. All walls of the world box (Left, Right, Back, Floor) belong to the WallLayer (8 in my case)
  - b. All LineEndPts belong to EndPtLayer (9 in my case)
  - c. All LineSegment belong to LineLayer (10 in my case)
  - d. With this separation, it is easy to only try to select types of objects with LMB
5. Implement and test each functionality *separately*. **DO NOT** implement new functionality in your final scene. With automatic generation of new TravelingBall and the strange orientation of geometries, it is simply impossible to test implementations. Create separate development scenes, test your implementations, ensure all is working and then integrate into the final scene (and you *will have bugs* during integration). At that point, go back to your separate development scenes to reproduce the bug and debug there. I *will not* help you debug your code in the final scene. I can't even do that for my own case.
6. Abstraction: AimLine, BigLine, these are objects of the same class (or derived class). You want to be able to abstract the behavior, and, create them into a set during runtime.
7. My implementation:
  - a. Create a scene for testing and manipulating **LineEndPts** (the walls must exist)
    - i. Click to select, drag to any where
  - b. Take step-a results and define the basic **LineSegment** class for **AimLine**
    - i. Make sure the line is properly drawn for all orientation of the end points
  - c. Take step-b results and define a simple **TravelingBall** class
    - i. Make sure balls can travel in the direction of the AimLine
  - d. Take step-c results and integrate sliders, test and verify

- i. Proper generation interval, proper speed, proper termination
- e. Take step-e results and define the **BigLine** (with proper understanding of AimLine, knowing how to instantiate new objects, this turned out to be a 10-minute step)
  - i. If your abstraction is done right, the only difference between a BigLine and AimLine is the line width
- f. Take step-f, support creation/deletion of new lines
  - i. This is actually not that challenging when everything is working

You should keep projects from each step, such that when bugs occur (not if, but when), you can go back to the previous simpler case, test find your bug and continue.

8. If you implement the AimLine as a prefab with spheres and cylinders as children then, you want:  
 travelingBall.localPosition = AimLine.Endpoint1.**position**;  
 // **NOT** AimLine.Endpoint1.**localPosition**, we will learn about this for MP5
9. Selecton: in my solution, I have three ray-cast layers: Wall, EndPoint, and Line. In my mouse-support, I iterate through the layers, e.g., when nothing is currently select, I always begin by casting in the EndPoint layer, if nothing selected, I then cast in the Wall layer, and I only try to delete when both of the previous did not ended up selecting anything. So, in many cases, I perform three separate ray-cast into different layers in one mouse-select update.

**Warning:** A global searched of the **RayCast()** function will be performed, **0% credit will be given to submissions** that call **RayCast()** anywhere other than in supporting mouse click user interaction.

#### Credit Distribution:

- |                                                           |             |            |
|-----------------------------------------------------------|-------------|------------|
| 1. <b>LineEndPt</b> mouse manipulation                    |             | <b>20%</b> |
| a. User can click and drag any of the LineEndPt on screen | 10%         |            |
| b. LineEndPt can be dragged to any of the walls           | 10%         |            |
| 2. <b>AimLine</b> and manipulation                        |             | <b>30%</b> |
| a. Correct line segment color/direction/dimension         | 10%         |            |
| b. Line properly begin/end at LineEndPt                   | 5%          |            |
| c. Click on Left Wall to create new instances             | 10%         |            |
| d. Click on line to delete the line                       | 5%          |            |
| e. <b>Line not interactively manipulatable</b>            | <b>-25%</b> |            |
| 3. <b>Traveling Ball</b>                                  |             | <b>25%</b> |
| a. Travel at proper direction                             | 10%         |            |
| b. Continuous spawning with the proper Interval           | 10%         |            |
| c. Travel at proper speed                                 | 10%         |            |
| 4. The <b>BigLine</b> _____                               |             | <b>20%</b> |
| a. Correct line segment color/direction/dimension         | 5%          |            |
| b. Line properly begin/end at LineEndPt                   | 5%          |            |
| c. Click on Back Wall to create new instances             | 5%          |            |
| d. Click on line to delete the line                       | 5%          |            |
| e. <b>Line not interactively manipulatable</b>            | <b>-20%</b> |            |
| 5. <b>Submission</b> proper zipfile name, EXE folder      |             | <b>5%</b>  |

This programming assignment counts 12% towards your final grade for this class.

**Warning:** You will receive an automatic zero for the entire assignment if you use RayCast to solve any of the given challenges (except select and create lines). Make sure to talk to me if you see the need to use RayCast in any other situation. There is a relatively small amount of code to be written, BUT, this is tricky. Start early! Let me know your questions as soon as you encounter any! **Remember**, Unity Physics engine, i.e., Kinematic, is completely off limit for this entire class, except in your final project. If you have any kinds of OnTriggerAnything() function, something is wrong!

**Extra Credit:** Some ideas, DO NOT work on the following until you are done with the base assignment. In all cases, you MUST support the default of just showing one AimLine. Otherwise, it can be impossible to test your submission.

In this MP, you can think of extra credits (according to your own interests) in (at least) three separate directions:

1. **Art/Design:** Are you interested in scene composition? Do you want to see MP3 in a more interesting setting (rather than the very boring set up)? Or,
2. **Math:** Are you interested in practicing (or demonstrating) your knowledge on math, then, maybe you can include a more interesting function than the boring line traveling. E.g., you can create traveling balls that follow a sinusoidal function along the aim line.